



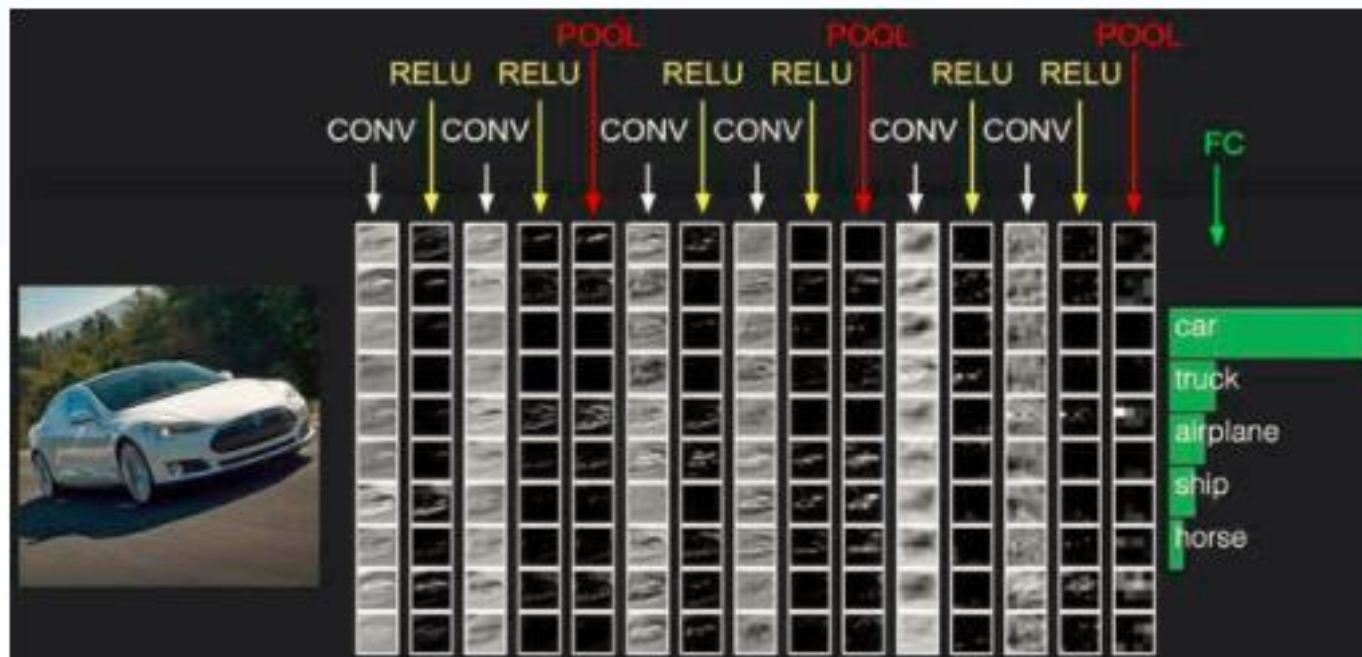
卷积神经网络

主讲人：吴昊

卷积神经网络

在深度学习领域中，作为被深度验证的成熟算法，深度卷积神经网络在图像识别、视频识别、语音识别领域取得了巨大的成功，正是由于这些成功，促成了当前深度学习乃至人工智能的大热。

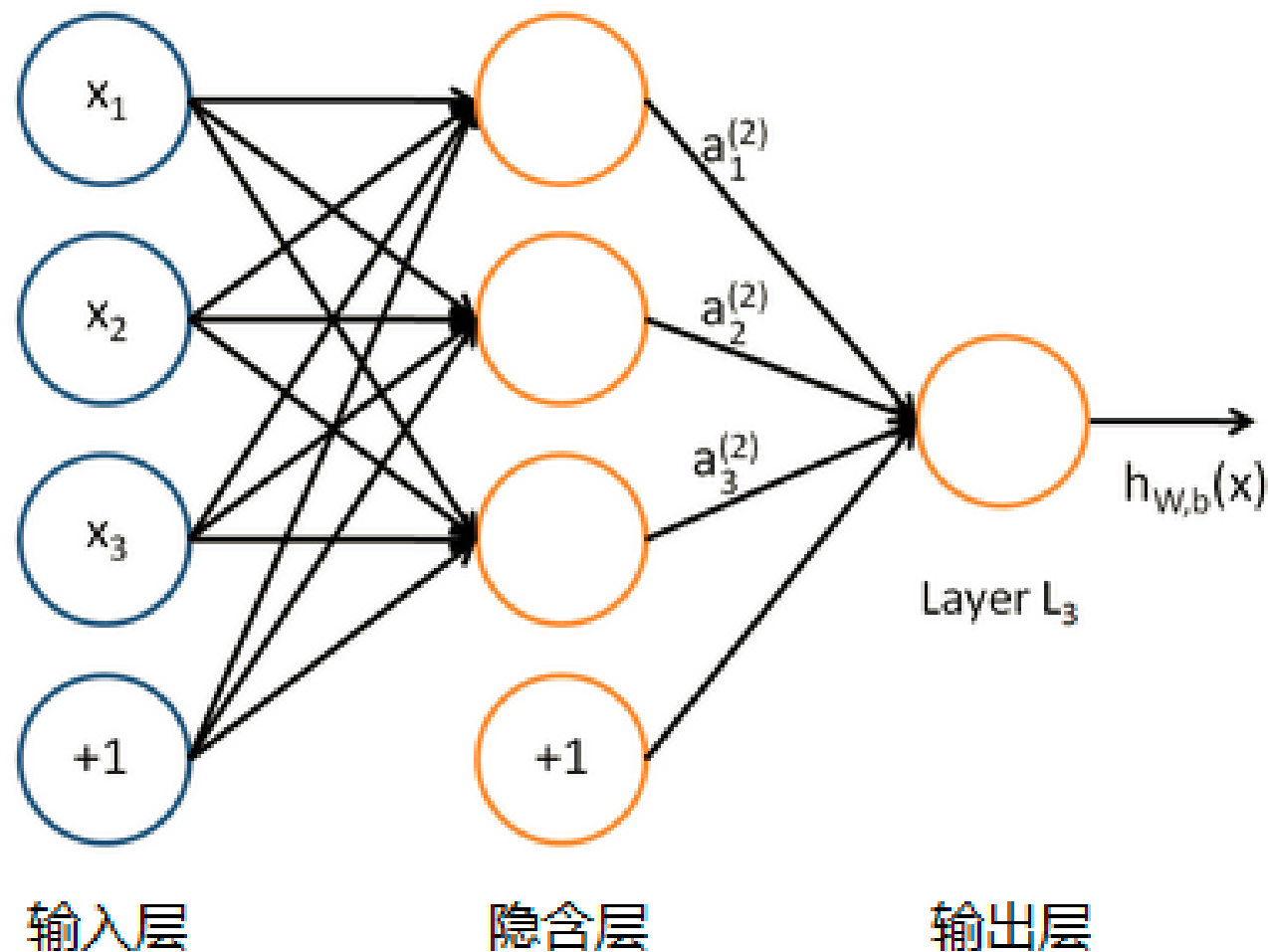
卷积神经网络（CNN），在计算机视觉、模式识别等领域主要利用一维、二维、三维神经网络完成处理、预测、识别等功能。一维卷积神经网络主要用于序列类的数据处理，二维卷积神经网络常应用于图像类信息的识别，三维卷积神经网络主要应用于视频类数据的识别。



神经网络

单个的感知器就构成了一个简单的模型，但在现实世界中，实际的决策模型则要复杂得多，往往是由多个感知器组成的多层网络。如右图所示，这也是经典的神经网络模型，由输入层、隐含层、输出层构成。

人工神经网络可以映射任意复杂的非线性关系，具有很强的鲁棒性、记忆能力、自学习等能力，在分类、预测、模式识别等方面有着广泛的应用。



经典神经网络和卷积神经网络

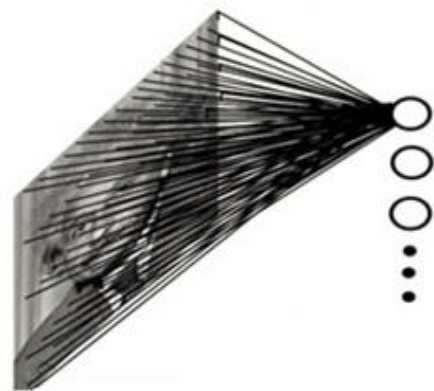
如果采用经典的神经网络模型，则需要读取整幅图像作为神经网络模型的输入（即全连接的方式），当图像的尺寸越大时，其连接的参数将变得很多，从而导致计算量非常大。

而我们对外界的认知一般是从局部到全局，先对局部有感知的认识，再逐步对全体有认知，这是人类的认识模式。在图像中的空间联系也是类似，局部范围内的像素之间联系较为紧密，而距离较远的像素则相关性较弱。

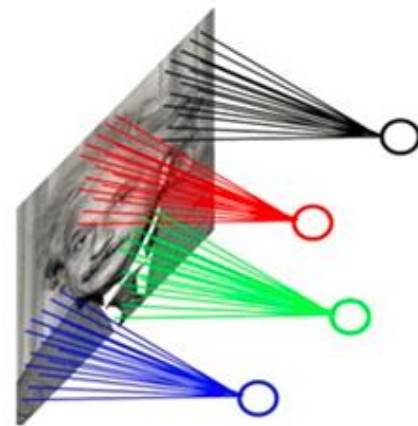
因而，每个神经元其实没有必要对全局图像进行感知，只需要对局部进行感知，然后在更高层将局部的信息综合起来就得到了全局的信息。这种模式就是卷积神经网络中降低参数数目的重要神器：局部感受野。

参考http://www.360doc.com/content/18/0318/09/47852988_738061922.shtml

全连接模式（经典神经网络）



局部连接模式（卷积神经网络）

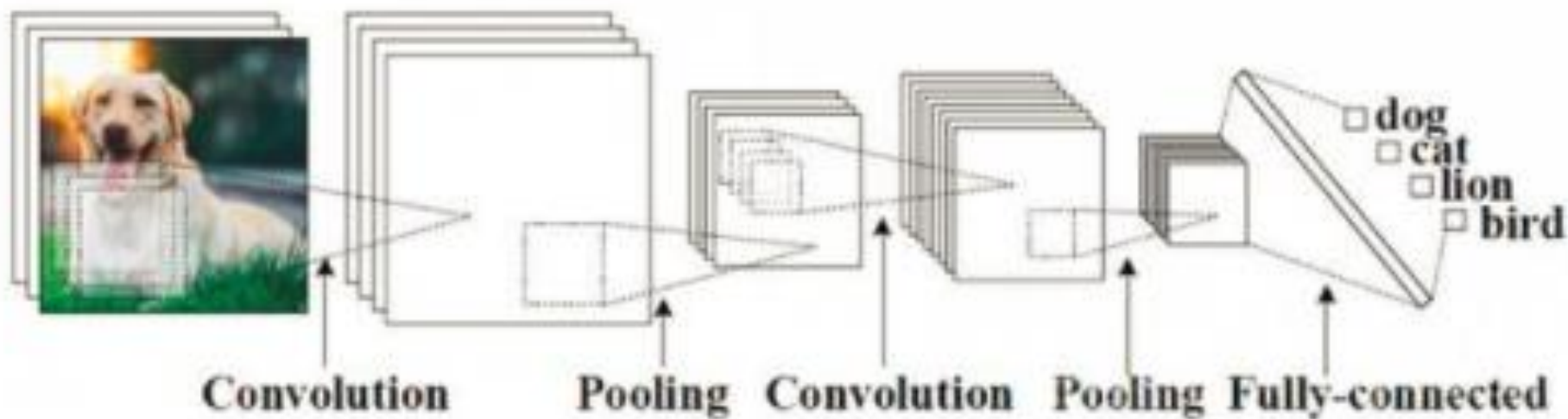


局部感受野

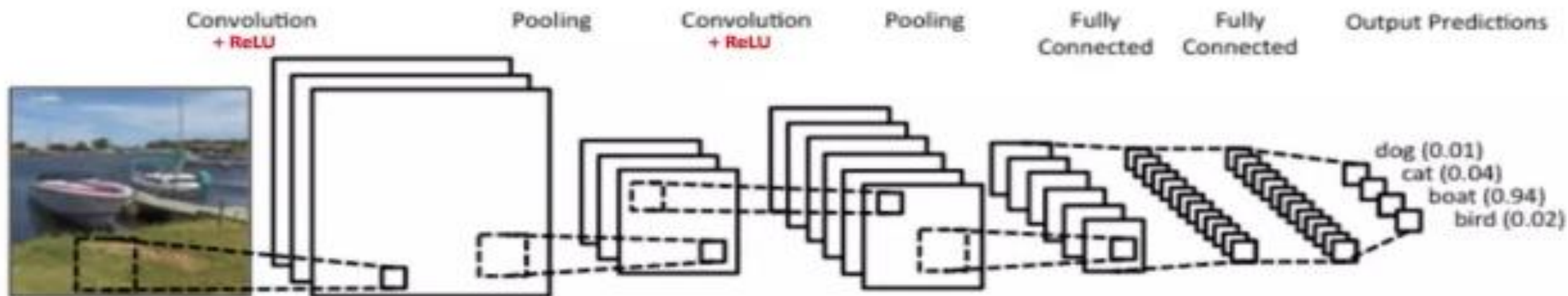
卷积神经网络

卷积神经网络是近些年发展起来的，并引起广泛重视的一种高效识别方法。20世纪60年代，科学家在研究猫脑皮层中用于局部敏感和方向选择的神经元时，发现其独特的网络结构可以有效地降低反馈神经网络的复杂性，继而提出了卷积神经网络（Convolutional Neural Networks-简称CNN）。

现在，CNN已经成为众多科学领域的研究热点之一，特别是在计算机视觉等领域。由于该网络避免了对图像的复杂前期预处理，可以直接输入原始图像，因而得到了更为广泛的应用。



卷积神经网络的基本构成



卷积神经网络 (Convolutional Neural Networks, CNN) 是一类包含卷积计算且具有深度结构的前馈神经网络 (Feedforward Neural Networks)，是深度学习 (deep learning) 的代表算法之一。

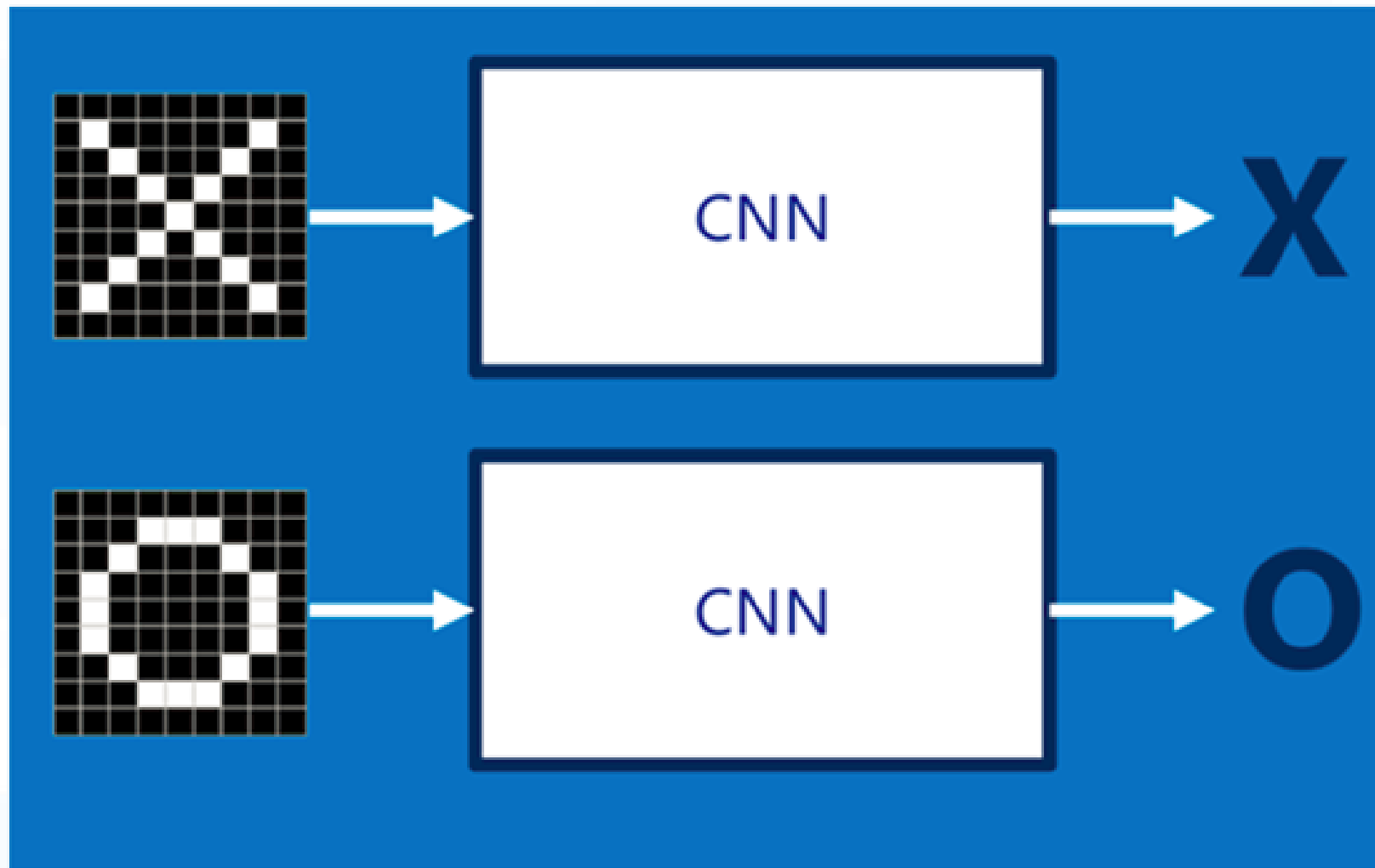
卷积神经网络中有四个主要操作：（1）卷积；（2）非线性变换；（3）池化或子采样；（4）分类（完全连接层）。这些操作是所有卷积神经网络的基本组成部分，因此了解它们的工作原理是理解卷积神经网络的重要步骤。下面我们将尝试直观地理解每个操作。

从上图示例可以看出，接收船只图像作为输入时，卷积神经网络在四个类别中正确地给船只分配了较高概率值 (0.94，输出层中所有概率的总和为1)，有非常好地识别、预测能力。

参考http://www.360doc.com/content/18/0409/00/7669533_744042884.shtml

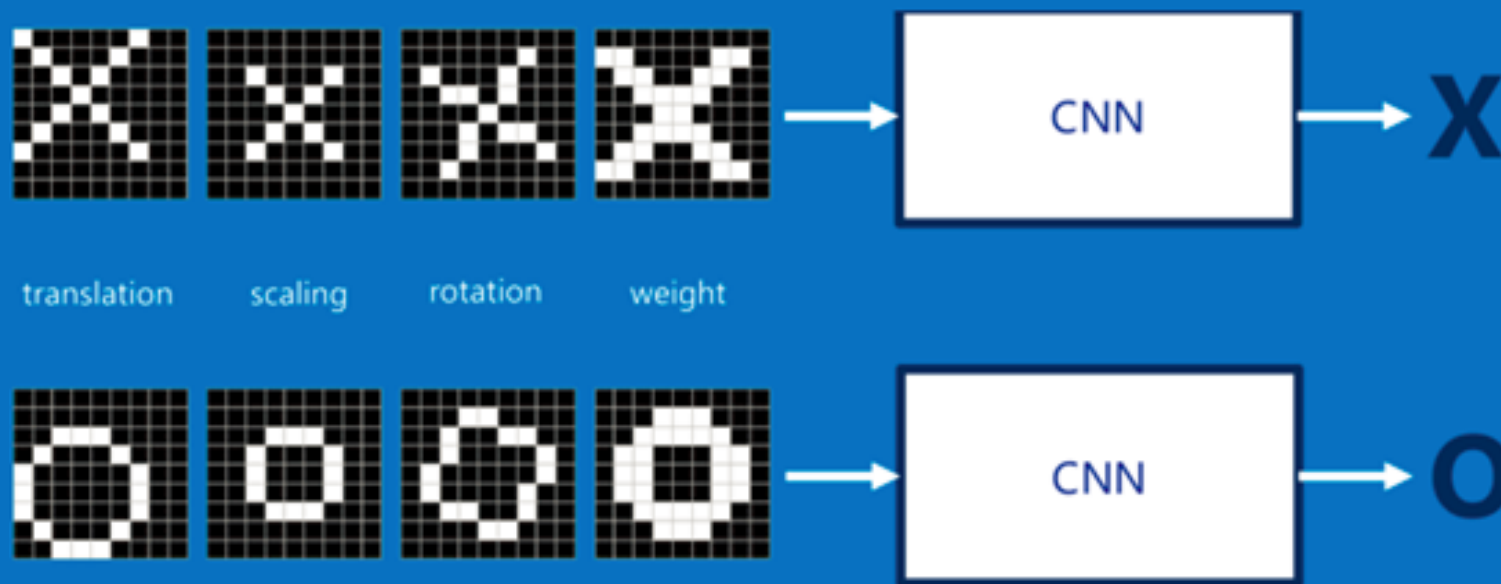
特征提取

假设给定一张图（可能是字母X或者字母O），通过CNN即可识别出是X还是O，如右图所示。具体是如何做到的呢？



特征提取

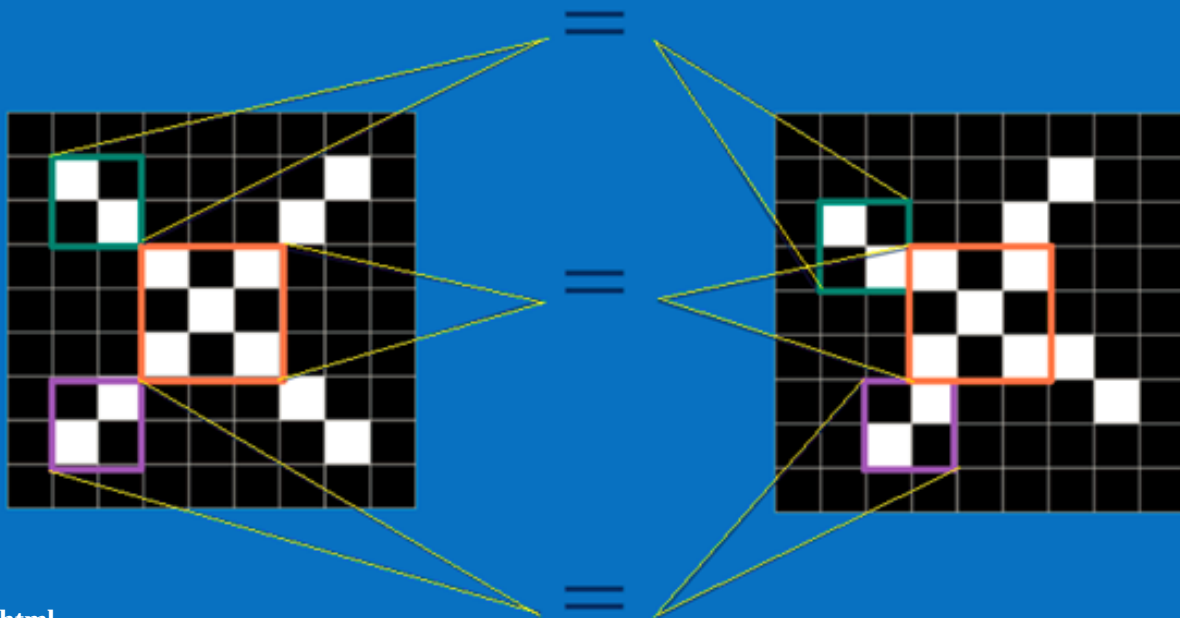
如果字母X、字母O是固定不变的，那么最简单的方式就是图像之间的像素一一比对就行。但在现实生活中，字体都有着各个形态上的变化（例如手写文字识别有平移、缩放、旋转、微变形等），如下图所示。



特征提取

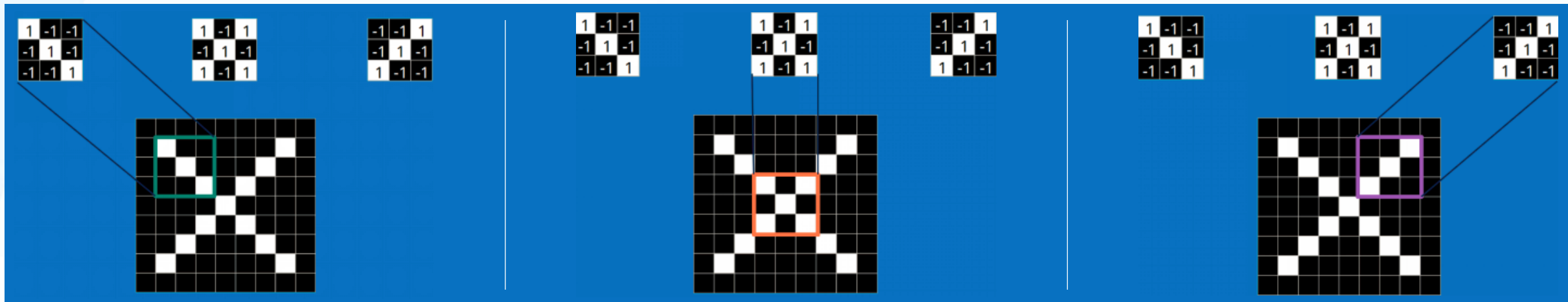
我们的目标是对于各种形态变化的X和O，都能通过CNN准确地识别出来，这就涉及到应该如何有效地提取特征，作为识别的关键因素。

前面叙述的“局部感受野”模式，对于CNN来说，它是小块地来进行比对，在两幅图像中大致相同的位置找到一些粗糙的特征（小块图像）进行匹配。相比起传统的整幅图逐一比对的方式，CNN的这种小块匹配方式能够更好地比较两幅图像之间的相似性。如下图：



特征提取

以字母X为例，可以提取出三个重要特征（两个对角区域、一个交叉区域），如下图所示：

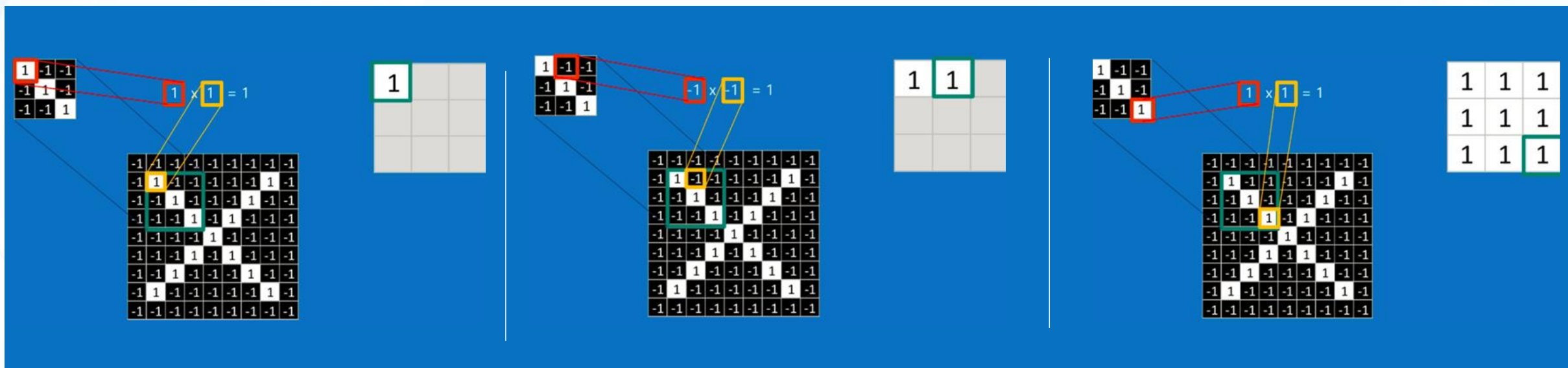


假如以像素值"1"代表白色，像素值"-1"代表黑色，则字母X的三个重要特征如下：

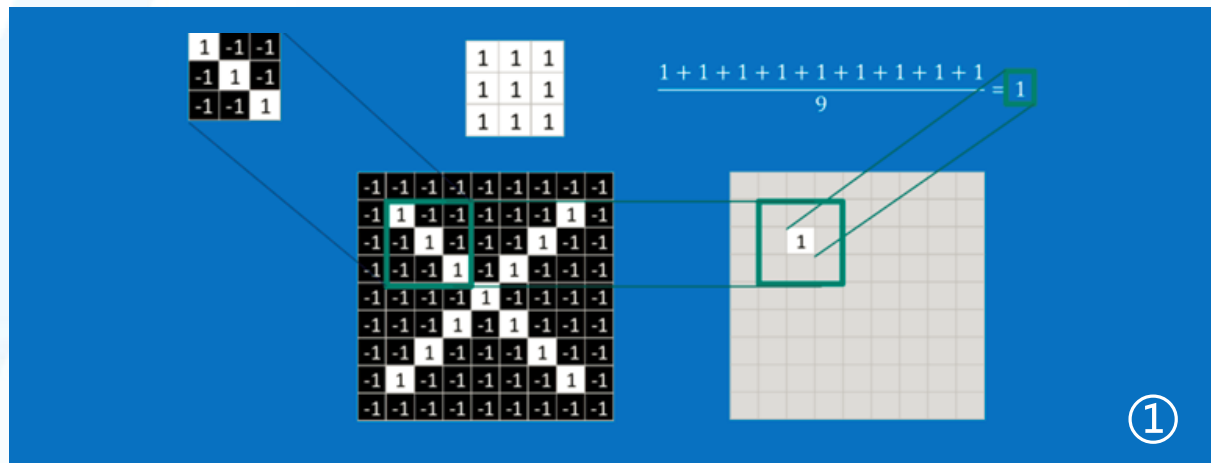


特征提取

如果两个像素点都是白色（值均为1），那么 $1*1 = 1$ ；如果均为黑色，那么 $(-1)*(-1) = 1$ 。也就是说，每一对能够匹配上的像素，其相乘结果为1。类似地，任何不匹配的像素相乘结果为-1。具体过程如下（第一个、第二个.....、最后一个像素的匹配结果）：

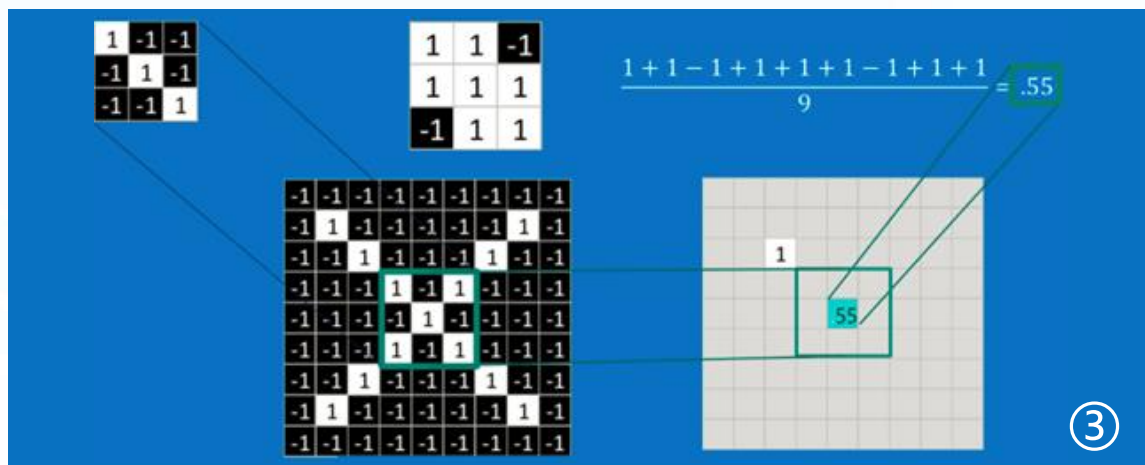
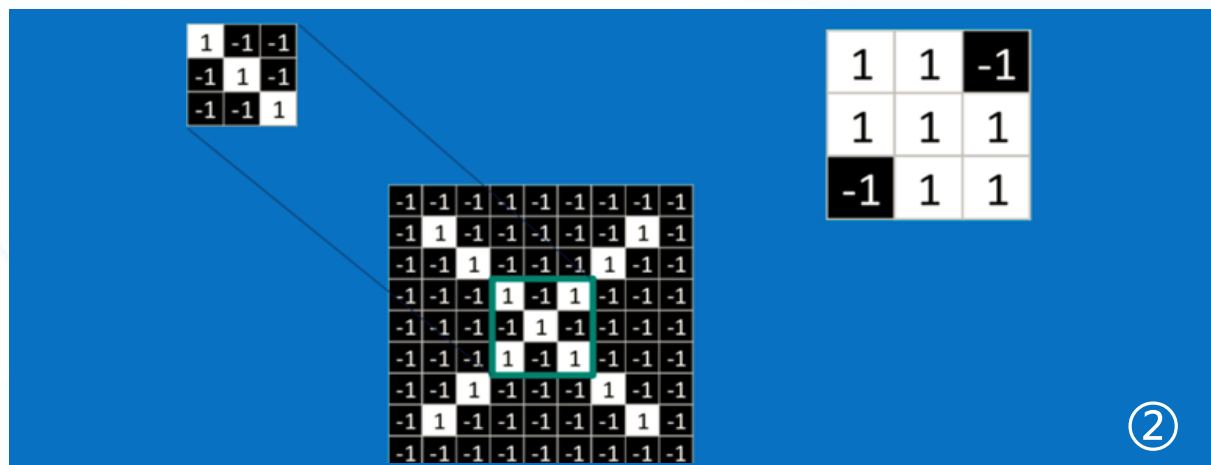


特征提取



根据卷积的计算方式，第一块特征匹配后的卷积计算如①图所示，结果为1。

对于其它位置的匹配，也是类似（例如中间部分的匹配如②图，计算之后的结果如③图。）



特征提取

基于上述思路，卷积可以实现特征的提取，卷积如何实现整幅图像的全局特征提取呢？

当给定一张新图时，CNN并不能准确地知道这些特征到底要匹配原图的哪些部分。所以，它会在原图中把每一个可能的位置都进行尝试，相当于把这个feature（特征）变成了一个过滤器。这个用来匹配的过程就被称为卷积操作，这也是卷积神经网络名字的由来。

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

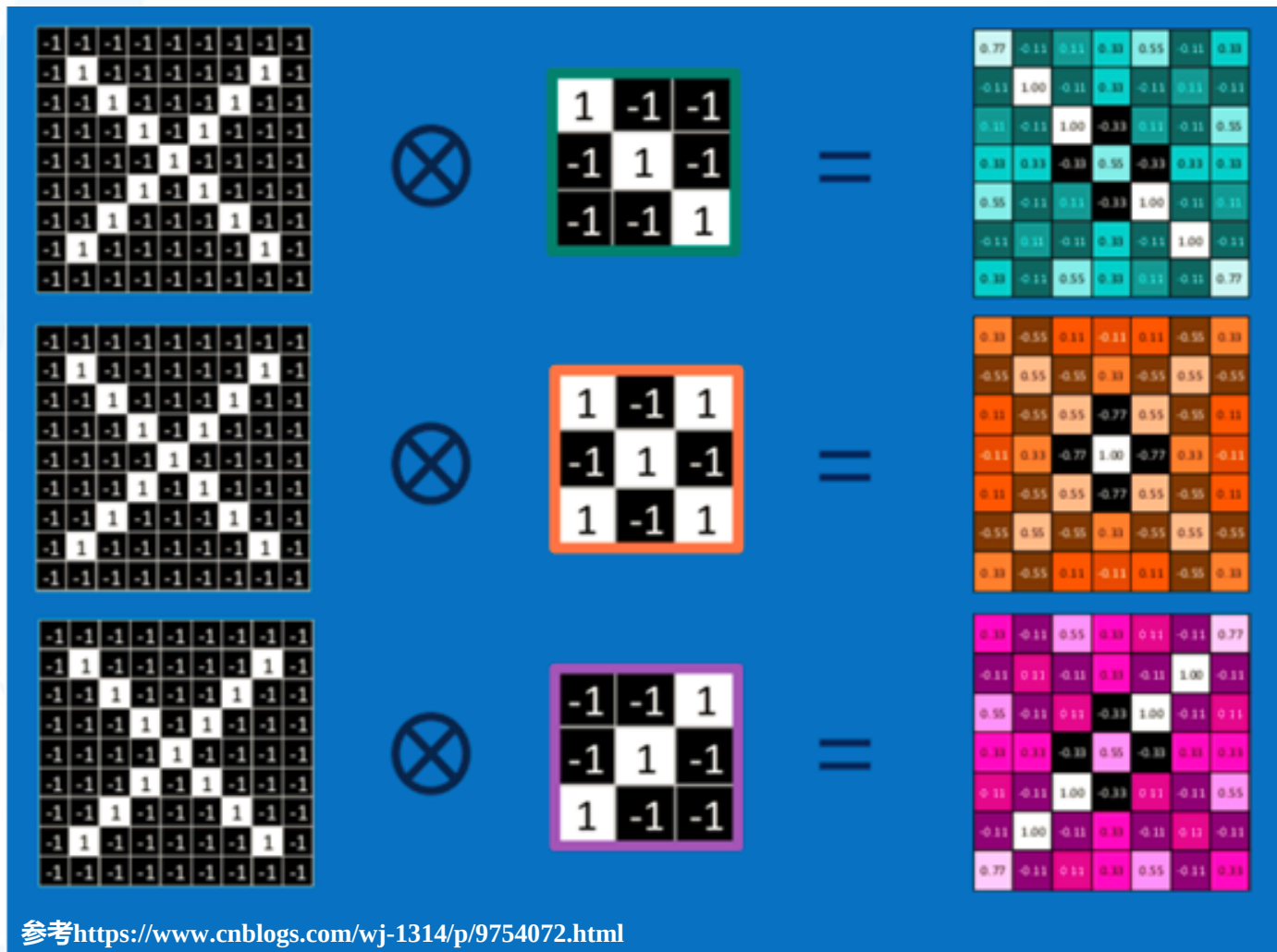
Image

4		

Convolved
Feature

卷积的操作如上图所示

特征提取



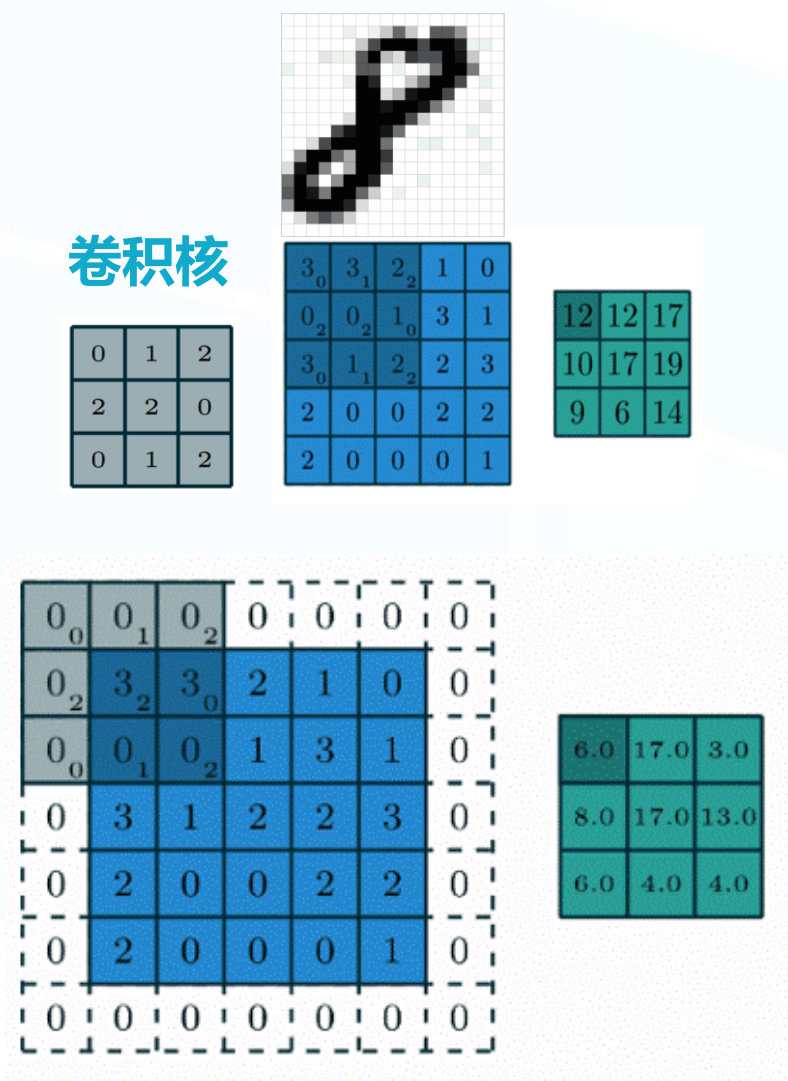
以此类推，利用三个特征区域不断地重复着上述过程。通过每一个feature（特征）的卷积操作，会得到一个新的二维数组，称之为feature map。其中的值，越接近1表示对应位置和feature的匹配越完整；越是接近-1，表示对应位置和feature的反向匹配越完整；而值接近0则表示对应位置和feature几乎没有任何匹配或者说没有什么关联。

可以看出，当图像尺寸增大时，其内部的加法、乘法和除法操作的次数会增加得很快。由于有这么多因素的影响，很容易使得计算量变得相当庞大。

特征提取

卷积在数学上用通俗的话来说就是输入矩阵与卷积核（卷积核也是矩阵）进行对应元素相乘并求和。所以一次卷积结果的输出是一个数，接下来对整个输入矩阵进行遍历，最终得到一个结果矩阵。

卷积在图像中的目的就是为了提取特征，这就是深度学习的核心内容之一。因为有了卷积层，才避免了我们来手动提取图像的特征，让卷积层自动提取图像高维度且有效的特征。虽然这没有经典特征提取方法（比如Canny边缘，SIFT，HOG等）的强大数学理论基础的支撑，但是卷积层提取的特征让最终的分类、识别结果非常理想。比如LeNet-5模型能在MNIST数据集上达到99%的识别率，一般来说网络结构越复杂、越深，往往最终的精确率会越高。



特征提取

对于同一张输入图像，不同的过滤器矩阵将会产生不同的特征映射。
例如，考虑如下输入图像：



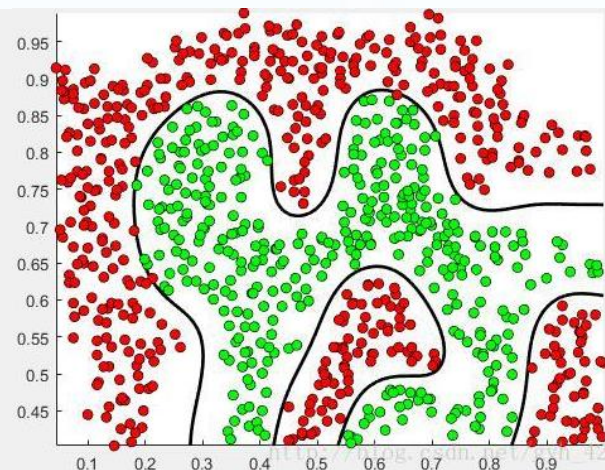
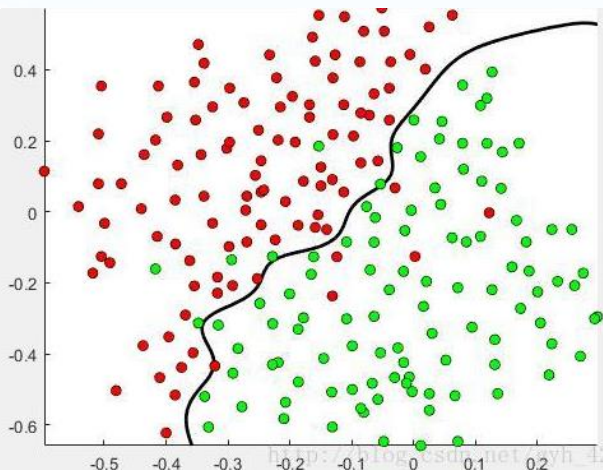
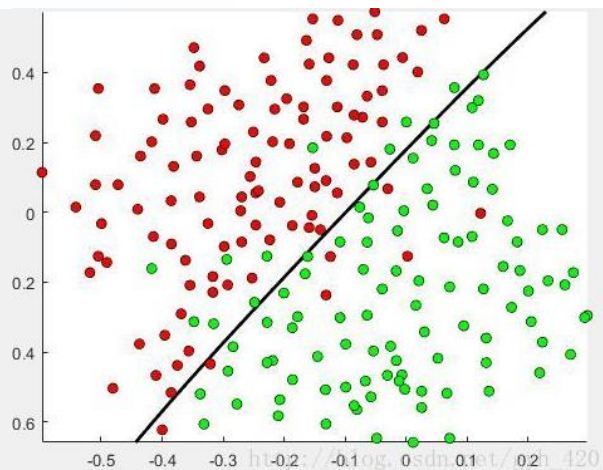
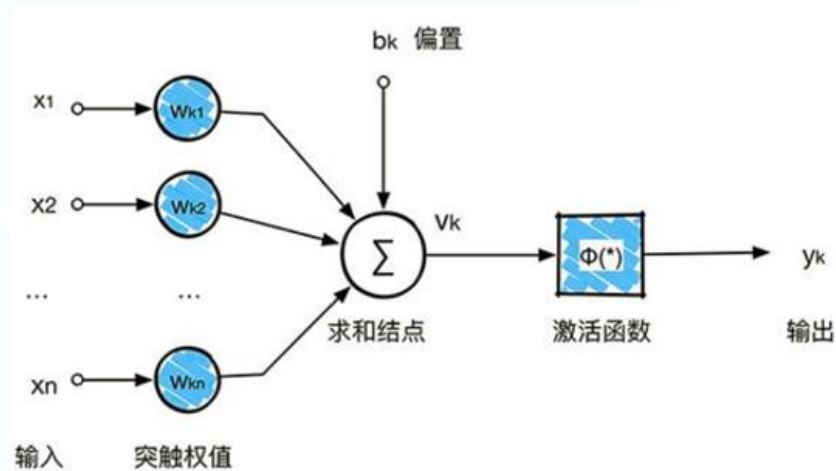
在右边，我们可以看到上图在不同过滤器下卷积的效果。如图所示，只需在卷积运算前改变过滤器矩阵的数值就可以执行边缘检测、锐化和模糊等不同操作。这意味着不同的过滤器可以检测图像的不同特征，例如边缘、曲线等。

参考https://blog.csdn.net/Mr_Robert/article/details/88987630

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

激活函数

激活函数一般是非线性函数。如果不用激活函数，每一层节点的输入都是上层输出的线性函数，多个感知机的组合得到的仍然是一个线性分类器。这样一来，我们就无法解决线性不可分的一些问题，包括简单的异或问题。因此，引入非线性函数作为激活函数，会使得深层神经网络几乎可以逼近任意函数。



参考<https://blog.csdn.net/qfikh/article/details/104798733>

激活函数

Sigmoid

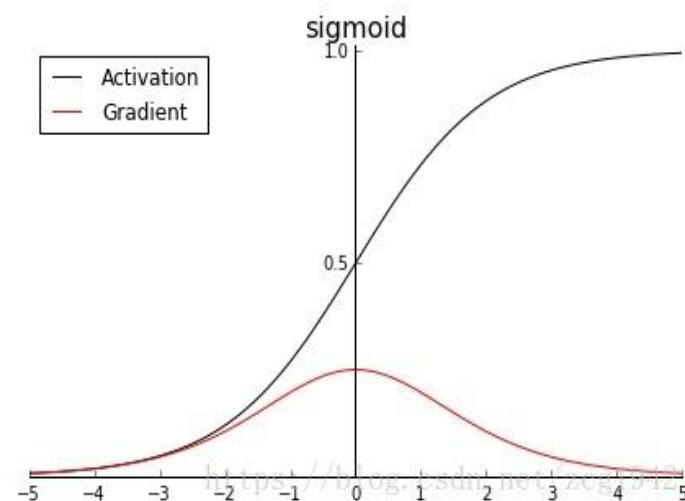
优点

能够把输入的连续实值变换为0和1之间的输出。

缺点

- ① 在深度神经网络中梯度反向传递时导致梯度爆炸和梯度消失。
- ② 输出不是零均值（即zero-centered）。
- ③ 解析式中含有幂运算，计算机求解时相对来讲比较耗时。对于规模比较大的深度网络，这会较大地增加训练时间。

$$\text{Sigmoid: } y(x) = \frac{1}{1+e^{-x}}$$



激活函数

Tanh

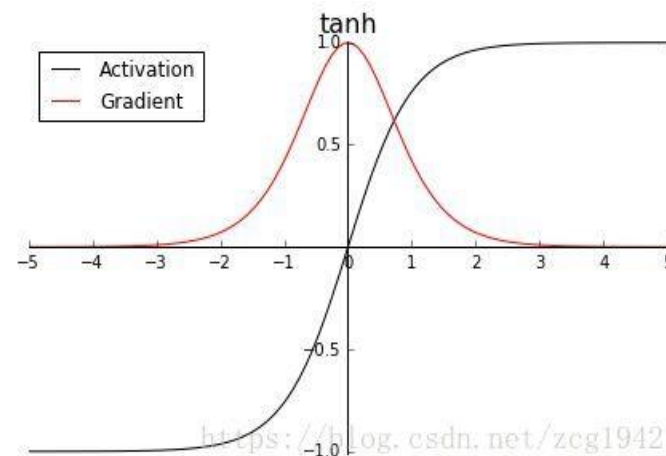
优点

解决了Sigmoid函数的不是zero-centered输出问题。

缺点

梯度消失和幂运算的问题仍然存在。

$$\text{Tanh}: y(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



激活函数

Relu

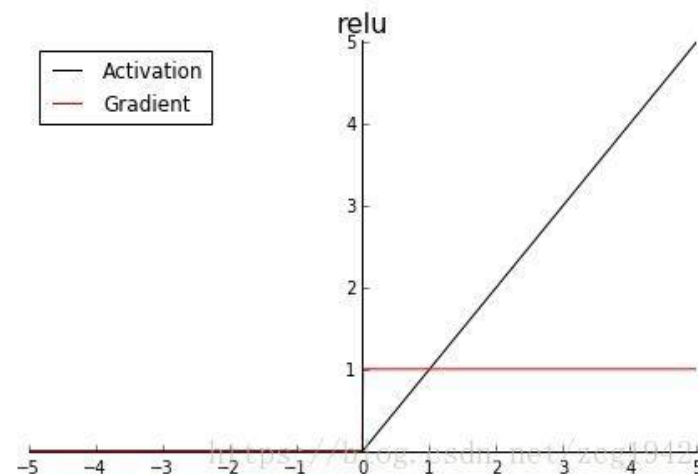
优点

- ① 在正区间解决了梯度消失问题。
- ② 计算速度非常快，只需要判断输入是否大于0。
- ③ 收敛速度远快于Sigmoid和Tanh。

缺点

- ① ReLU 的输出不是 zero-centered。
- ② Dead ReLU Problem: 指的是在输入为负数的时候激活函数直接为0，造成所谓的神经元坏死，不能给之后的神经元传递信息。

$$\text{Relu: } y(x) = \max(0, x)$$



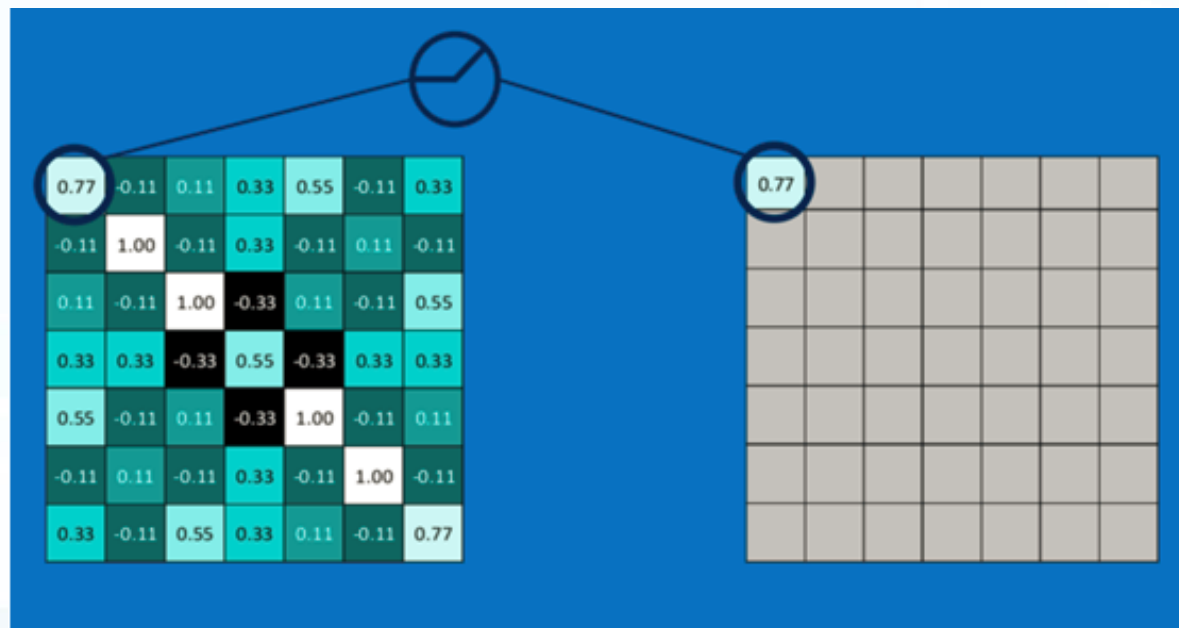
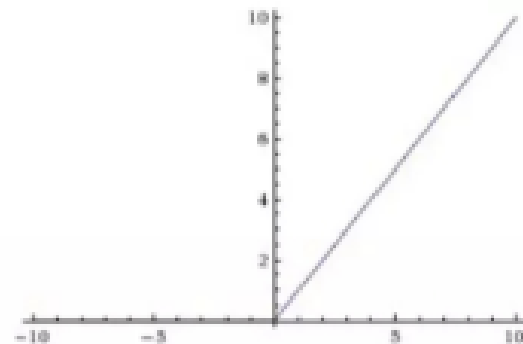
激活函数

如前面所述，常用的激活函数有Sigmoid，Tanh，Relu等。由于Relu有收敛快、求梯度简单等特点，故卷积神经网络中的激活函数一般使用ReLU。Sigmoid，Tanh有时也会应用于全连接层。

计算公式比较简单，即对于输入的负值，输出全为0；对于正值，则原样输出。

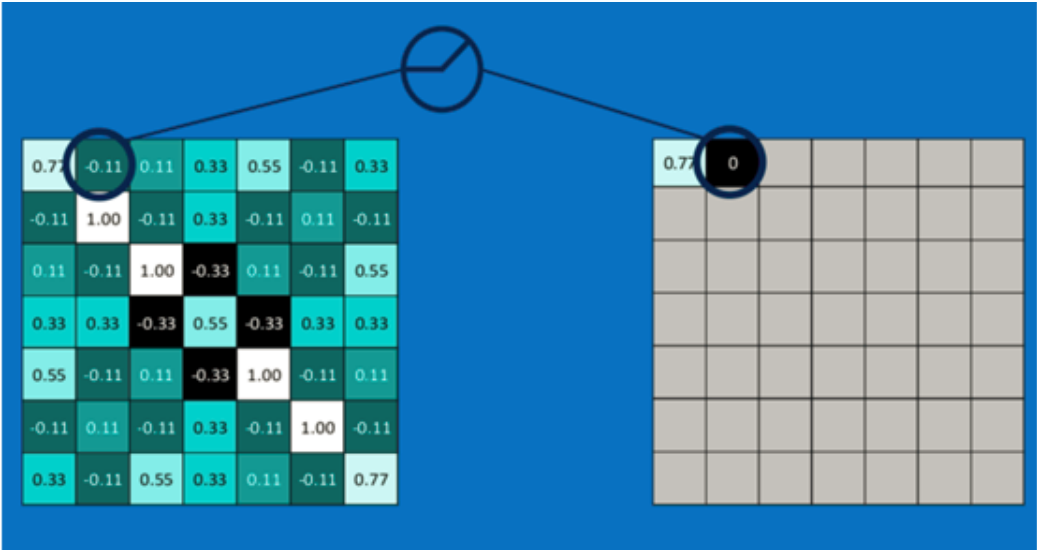
下面看一下ReLU激活函数操作过程：第一个值，取 $\max(0, 0.77)$ ，结果为0.77，如右图。

$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$

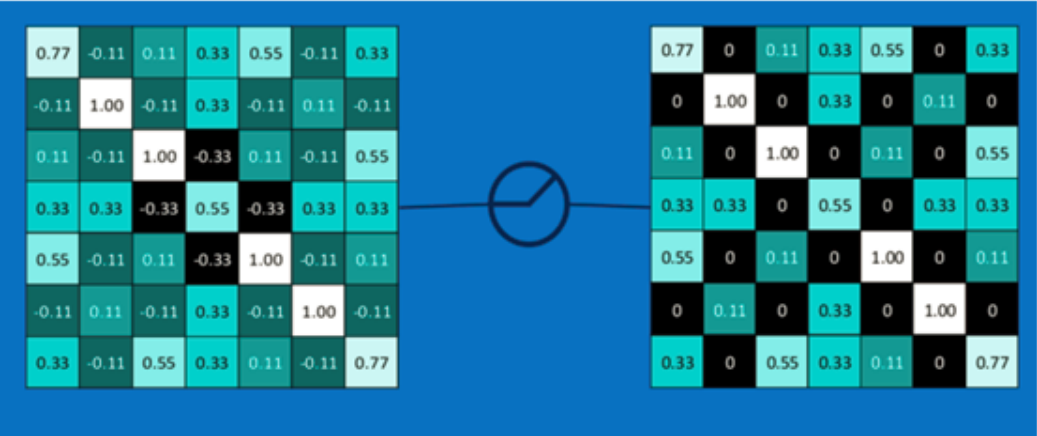


激活函数

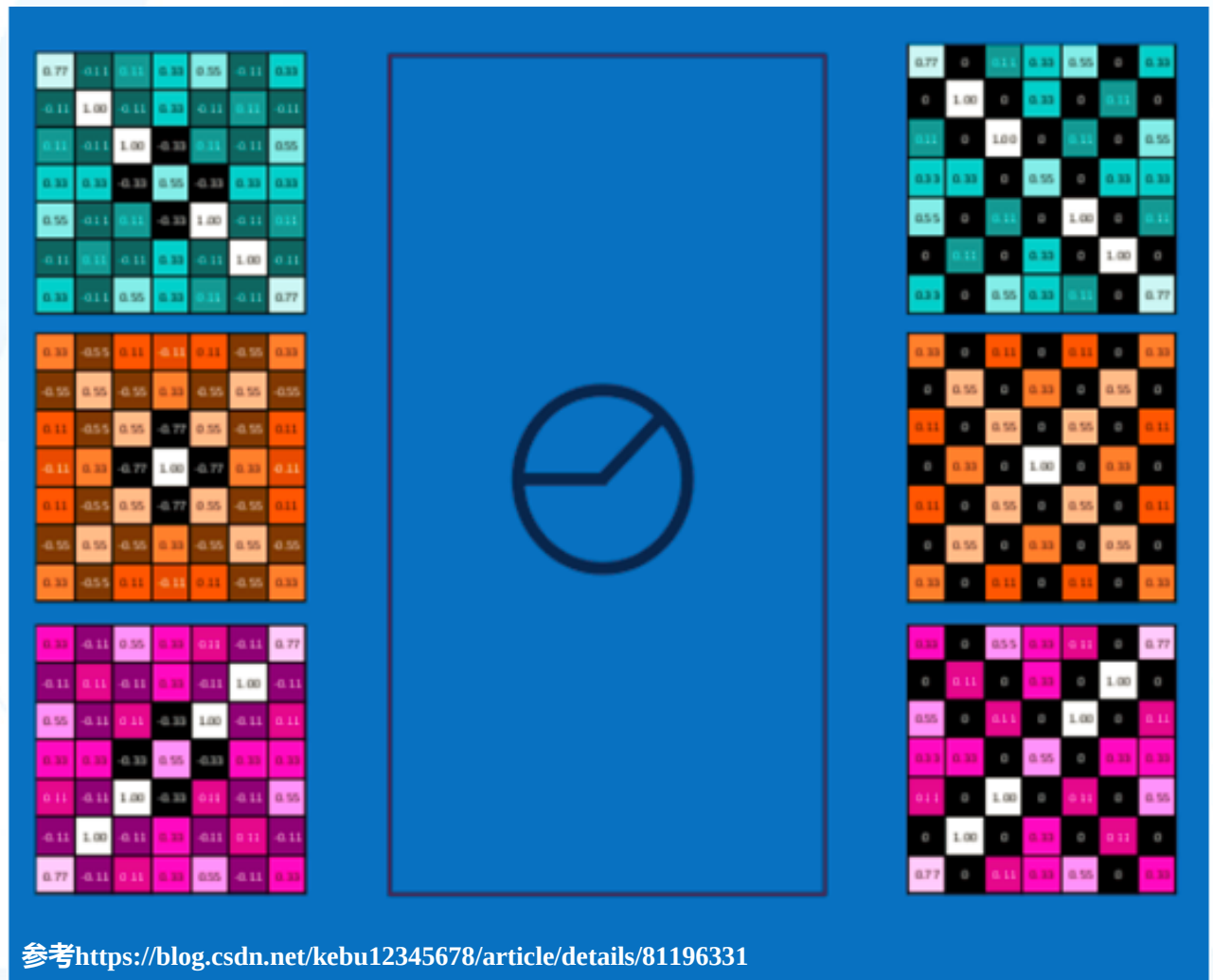
第二个值，取 $\max(0, -0.11)$ ，结果为0，如右图：



以此类推，整个图经过ReLU激活函数运算之后，结果如右图：

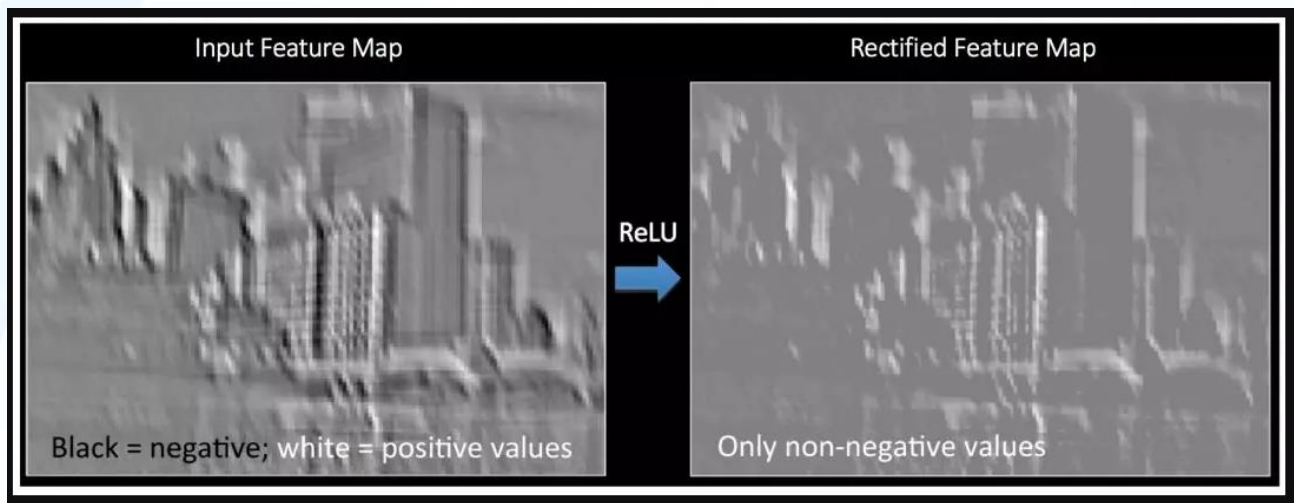


激活函数



对所有的feature map执行
ReLU激活函数操作，结果如左图。

激活函数



左侧的示意图可以很清楚地理解 ReLU 操作，它展示了将 ReLU 作用于图中某个特征映射得到的结果。这里的输出特征映射也被称为“修正”特征映射。

其它非线性函数诸如 Tanh 或 Sigmoid 也可以用来代替 ReLU，但是在大多数情况下，ReLU 的表现更好。

池化

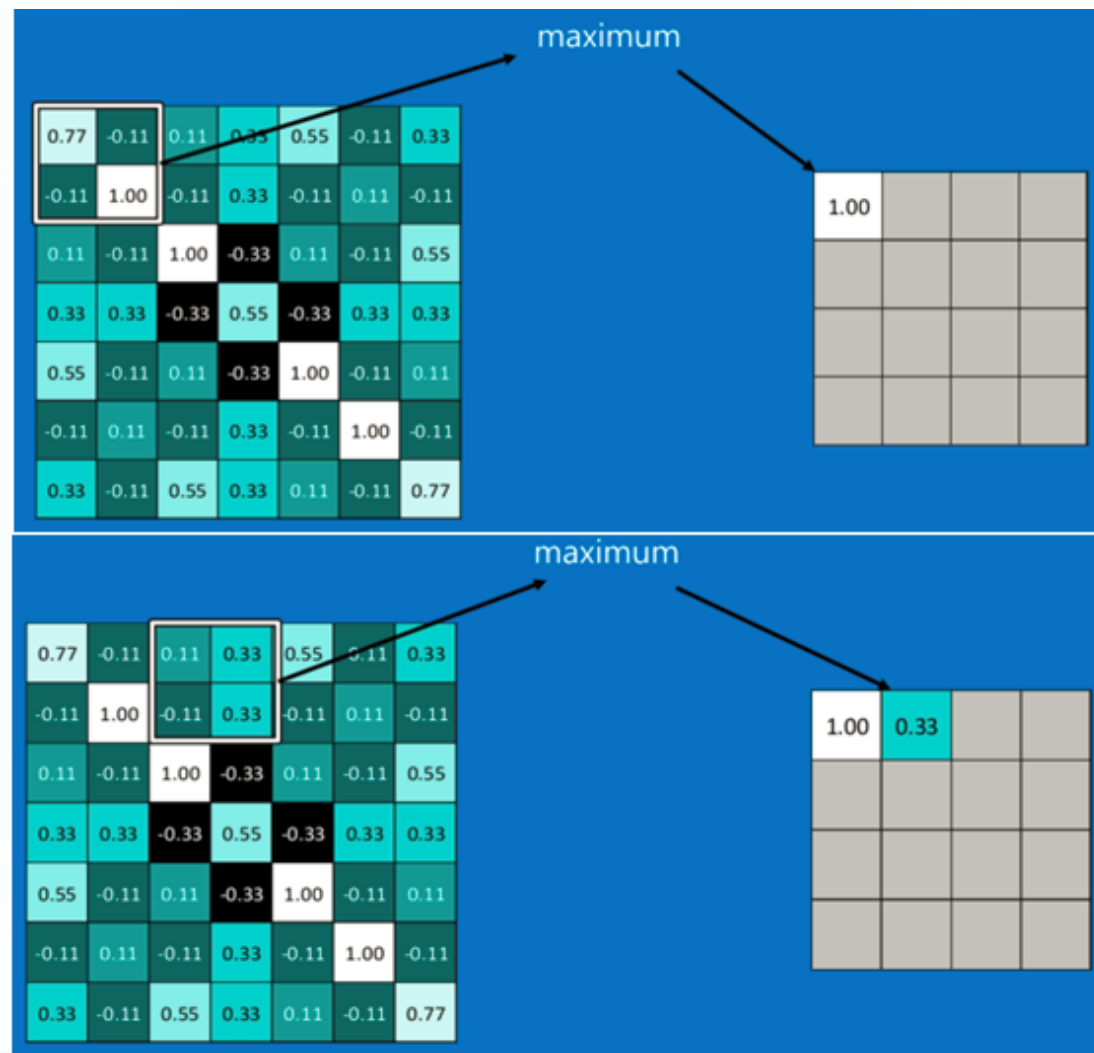
为了有效地减少计算量，CNN使用的另一个有效的工具被称为“池化(Pooling)”。池化就是将输入图像进行缩小，减少像素信息，只保留重要信息。

池化的操作也很简单。通常情况下，池化区域是2*2大小，然后按一定规则转换成相应的值。例如取这个池化区域内的最大值（max-pooling）、平均值（mean-pooling）等。

右上图显示了左上角2*2池化区域的max-pooling结果，取该区域的最大值 $\max(0.77, -0.11, -0.11, 1.00)$ ，作为池化后的结果。

池化区域往右，第二小块取大值 $\max(0.11, 0.33, -0.11, 0.33)$ ，作为池化后的结果，如右下图所示。

参考<https://blog.csdn.net/kebu12345678/article/details/81196331>



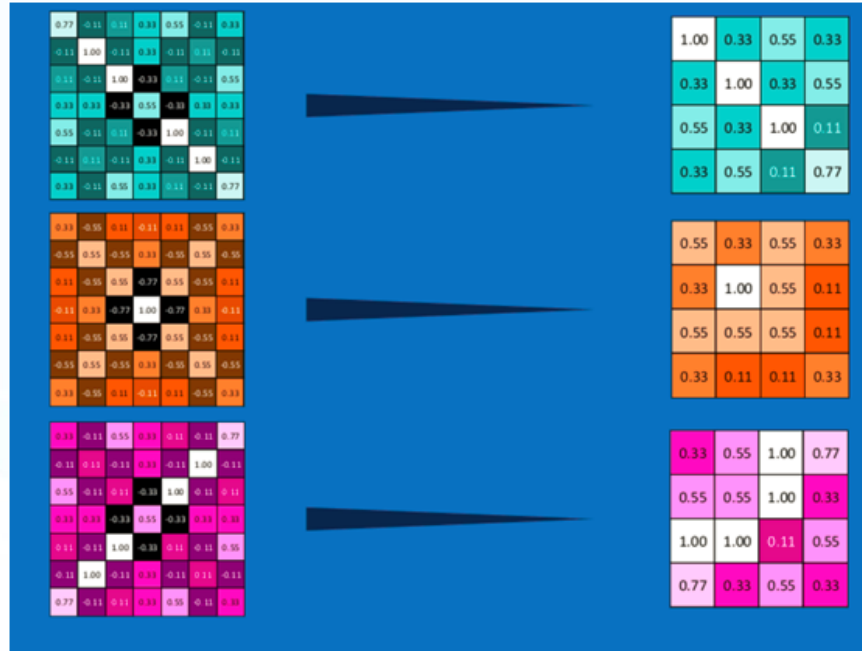
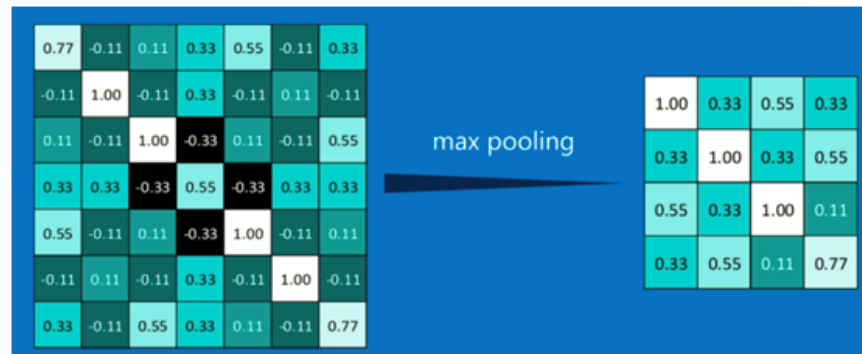
池化

其它区域也是类似，取区域内的最大值作为池化后的结果，最后的结果如右图所示。

最大池化（max-pooling）保留了每一小块内的最大值，也就是相当于保留了这一块最佳的匹配结果（因为值越接近1表示匹配越好）。它不会具体关注窗口内到底是哪一个地方匹配了，而只关注是不是有某个地方匹配上了。

通过加入池化层，图像尺寸缩小了，能很大程度上减少计算量、降低机器负载。

参考<https://blog.csdn.net/kebu12345678/article/details/81196331>



池化

池化操作的过程和卷积很类似，但是卷积是用来提取特征的，池化层是用来减少卷积层提取的特征个数的，可以理解为是用来增加特征的鲁棒性或者是降维。

<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>3.0</td><td>3.0</td></tr><tr><td>3.0</td><td>2.0</td><td>3.0</td></tr></table>	3.0	3.0	3.0	3.0	3.0	3.0	3.0	2.0	3.0
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3.0	3.0	3.0																																																																																																									
3.0	3.0	3.0																																																																																																									
3.0	2.0	3.0																																																																																																									

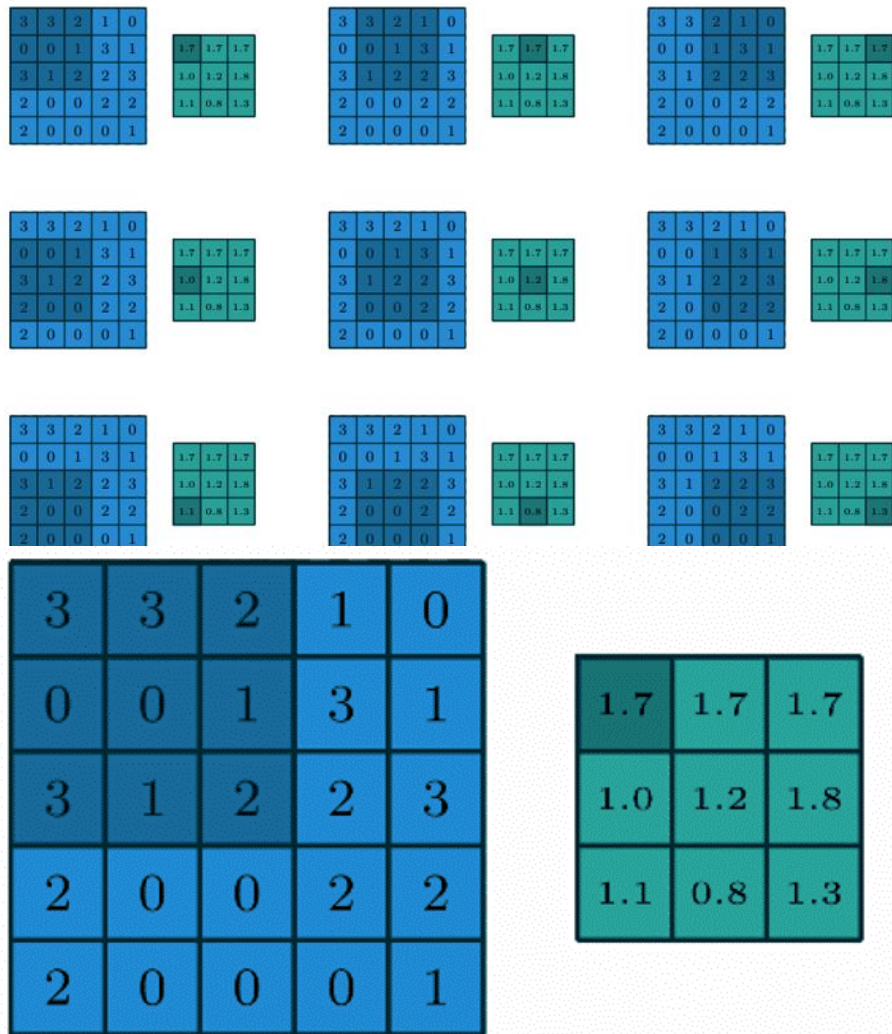
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

空间池化（也称为子采样或下采样）可降低每个特征映射的维度，并保留最重要的信息。空间池化有几种不同的方式：较大值、平均值、求和等。

在实际运用中，较大池化的表现更好。分析一下为何较大池化表现更好？

池化

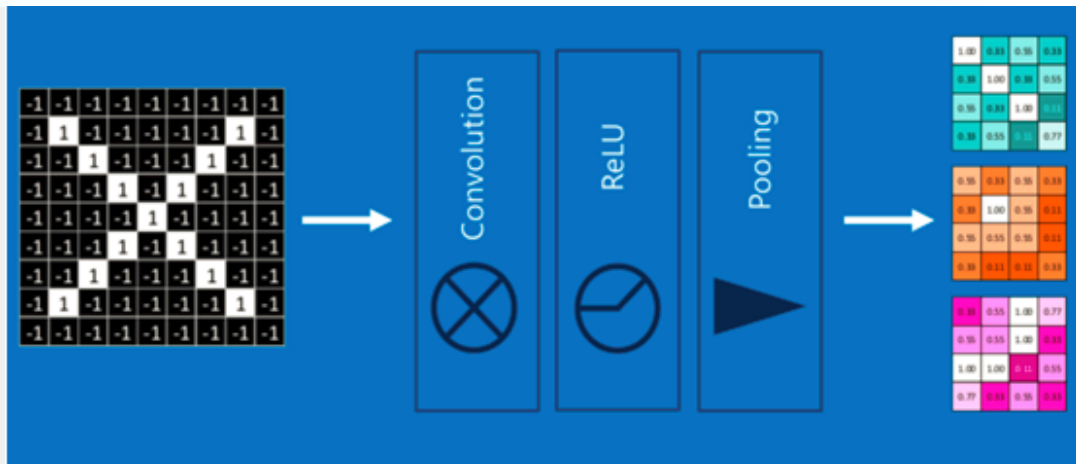


优点

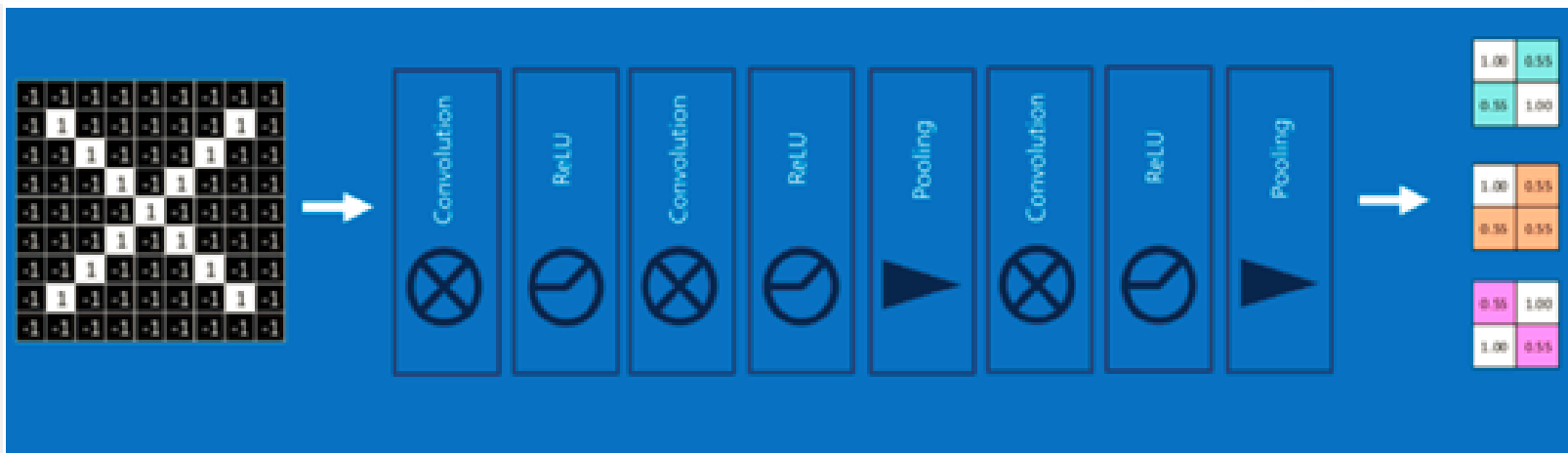
- ① 使输入（特征维度）更小，更易于管理。
- ② 减少网络中的参数和运算次数，因此可以控制过拟合。
- ③ 使网络对输入图像微小的变换、失真和平移更加稳健（输入图片小幅度的失真不会改池化的输出结果——因为我们取了邻域的较大值/平均值）。
- ④ 可以得到比例等变的图像。这是非常有用的，这样无论图片中的物体位于何处，我们都可以检测到。

池化

将上面所提到的卷积、激活函数、池化组合在一起，就变成右图：



通过加大网络的深度，增加更多的层，就得到了深度神经网络，如右图：



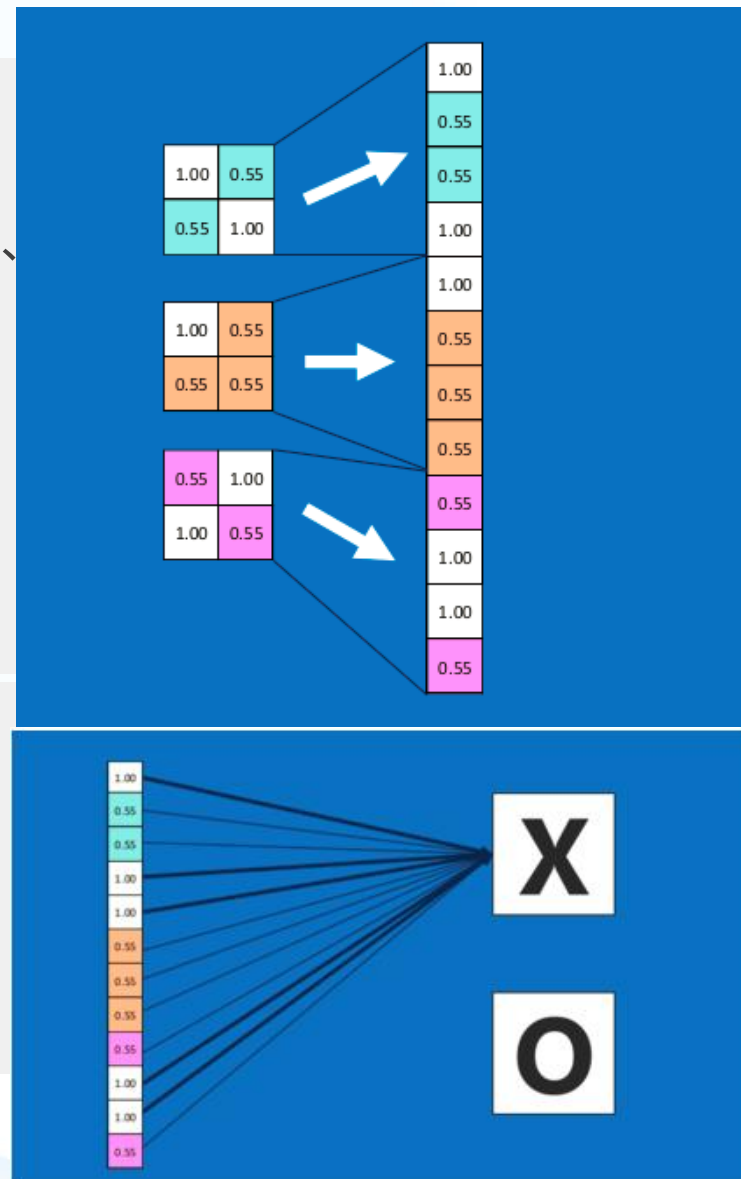
全连接层(Fully connected layers)

全连接层在整个卷积神经网络中起到“分类器”的作用，即通过卷积、激活函数、池化等深度网络后，再经过全连接层对结果进行识别分类。

首先将经过卷积、激活函数、池化的深度网络后的结果串接起来，如右图所示：

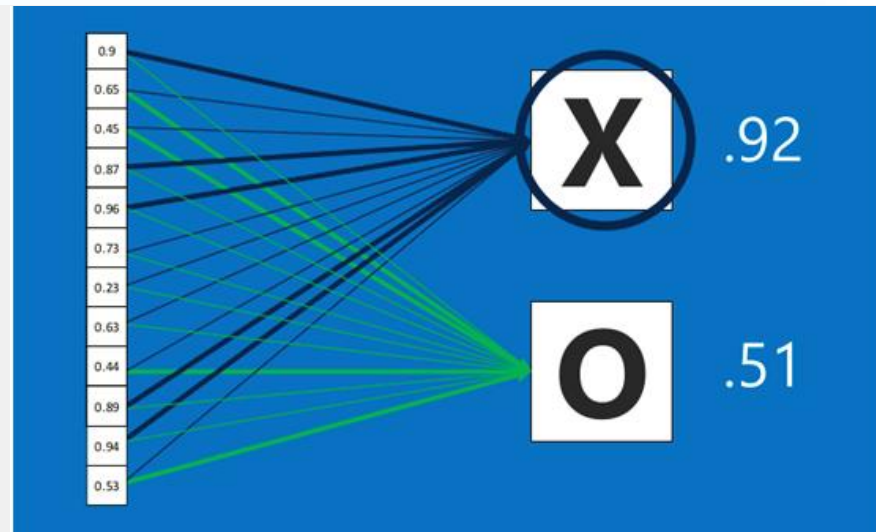
由于神经网络属于监督学习，故在模型训练时，我们根据训练样本对模型进行训练，从而得到全连接层的权重（如预测字母X的所有连接的权重）

参考<https://blog.csdn.net/kebu12345678/article/details/81196331>

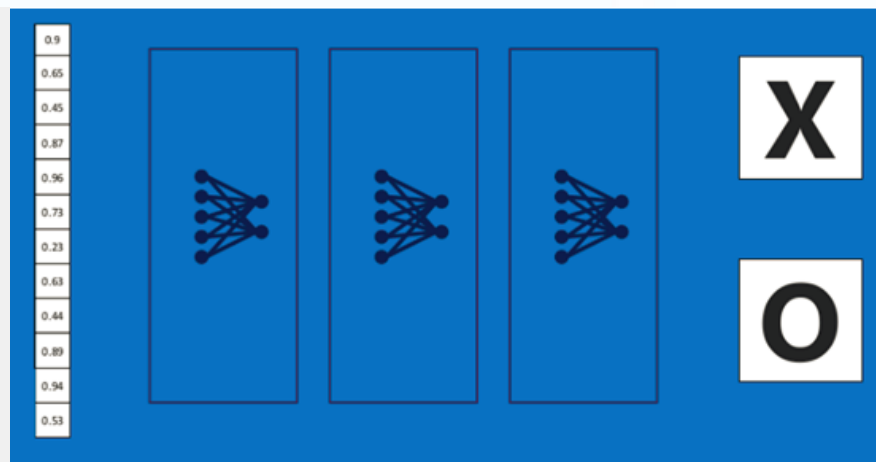


全连接层

在利用该模型进行结果识别时，把模型训练得出来的权重以及经过前面的卷积、激活函数、池化等过程计算出来的结果进行加权求和，得到各个结果的预测值，然后取值最大的作为识别的结果（如右图，最后计算出来字母X的识别值为0.92，字母O的识别值为0.51，则结果判定为X）



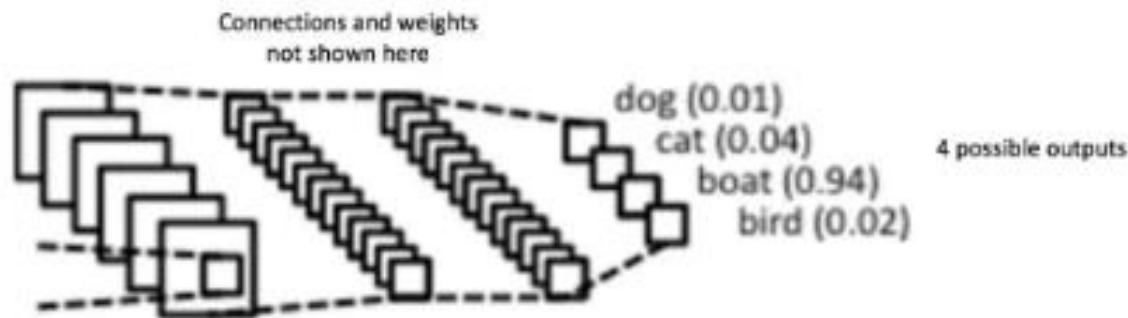
上述这个过程定义的操作为“全连接层” (Fully connected layers), 全连接层也可以有多个，如右图：



全连接层

完全连接层是一个传统的多层感知器，它在输出层使用 softmax 激活函数（也可以使用其它分类器，比如 SVM，但在这里只用到了 softmax）。“完全连接”这个术语意味着前一层中的每个神经元都连接到下一层的每个神经元。

卷积层和池化层的输出代表了输入图像的高级特征。完全连接层的目的是利用这些基于训练数据集得到的特征，将输入图像分为不同的类。例如，我们要执行的图像分类任务有四个可能的输出，如下图所示：



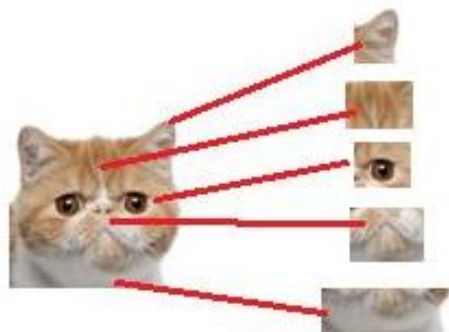
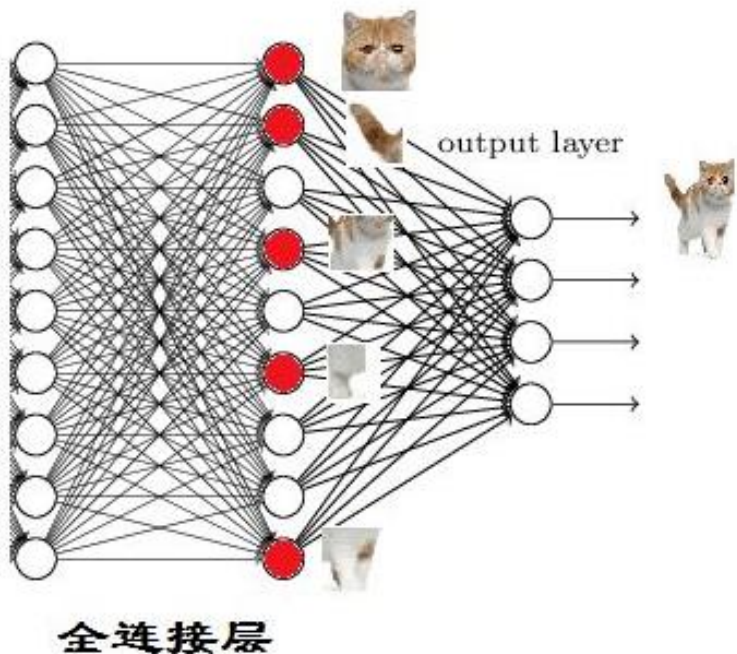
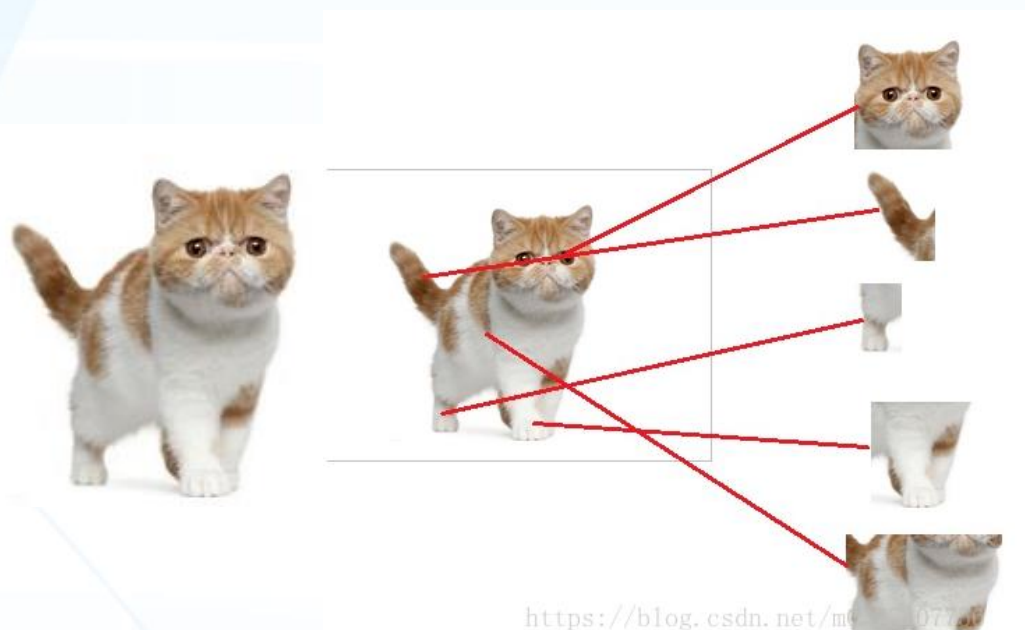
除分类之外，添加完全连接层也是一个（通常来说）比较简单的学习这些特征非线性组合的方式。卷积层和池化层得到的大部分特征对分类的效果可能也不错，但这些特征的组合可能会更好。

完全连接层的输出概率之和为1，这是因为我们在完全连接层的输出层使用了softmax函数。Softmax函数取任意实数向量作为输入，并将其压缩为数值在0到1之间、总和为1的向量。

参考<https://www.cnblogs.com/kkx5211/p/11884267.html>

全连接层

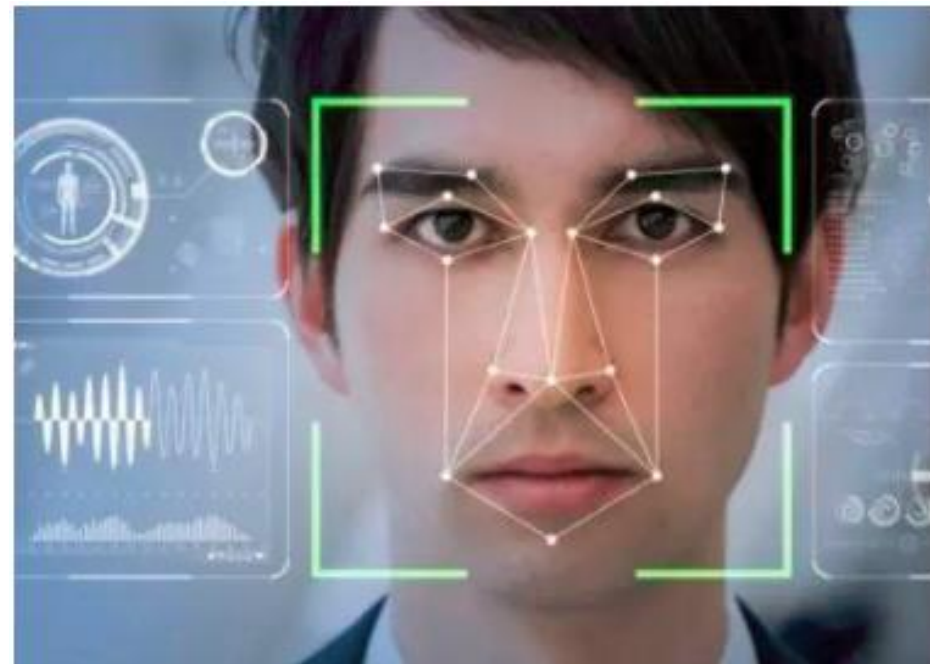
全连接层之前模块的作用是提取特征，全连接层本身的作用是分类，我们现在的任务是去区分一幅图片是不是猫。当我们把这些找到的特征组合在一起，发现最符合要求的是猫。



特征组合

图像的底层特征与特征在图片的位置关系不显著。在做人脸识别的时候，眼角、嘴角等特征不需要充分的考虑其它“五官”就可以提取出来。这样一来，无论原来的图像矩阵有多大，在局部用 $F \times F$ 的卷积核提取局部特征就可以。一组 $F \times F$ 的参数能得到一张 feature map, 一般会有多个卷积核，分别经过卷积后就可以有好几层 feature map。

但是对于高层级的特征，一般是和位置有关的，比如一张人脸图片，眼睛、鼻子、嘴巴的是有位置和权重关系的，这个时候卷积层就无法胜任高层次特征提取，需要局部连接层或者全连接层将低层特征进行深度组合。举个反例，如果输入是五官排布错位的“人脸”，也会被电脑错误地识别为人脸。

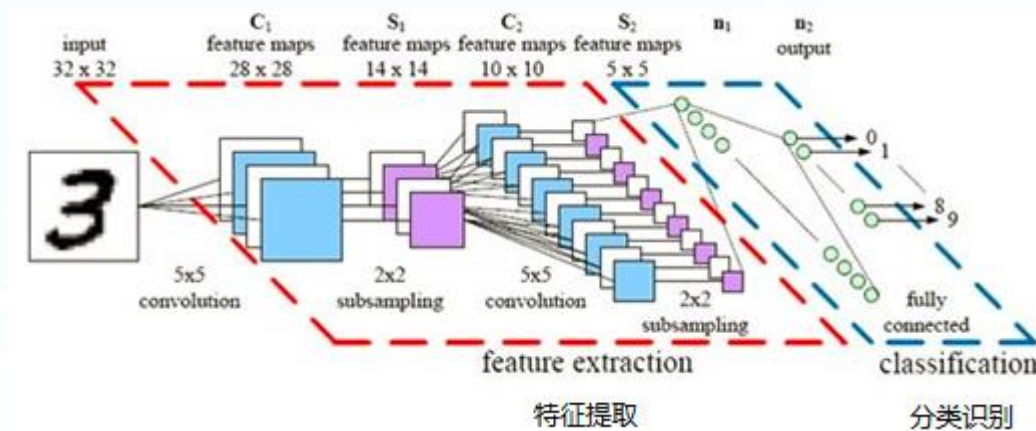


组合网络

将以上所有模块（含结果）串起来后，就形成了一个“卷积神经网络”（CNN）结构，如下图所示：



最后，再回顾总结一下，卷积神经网络主要由两部分组成：一部分用来实现特征提取（卷积、激活函数、池化），另一部分用来实现分类识别（全连接层）。右图便是著名的手写文字识别卷积神经网络结构图。



反向传播

如前文所述，卷积+池化层用来从输入图像提取特征，完全连接层用来做分类器。下面，利用反向传播对网络进行训练。

下图中，由于输入图像是船，故船类目标概率为1，其他三个类为0。

卷积网络的整体训练过程概括如下（目标向量 = $[0, 0, 1, 0]$ ）：

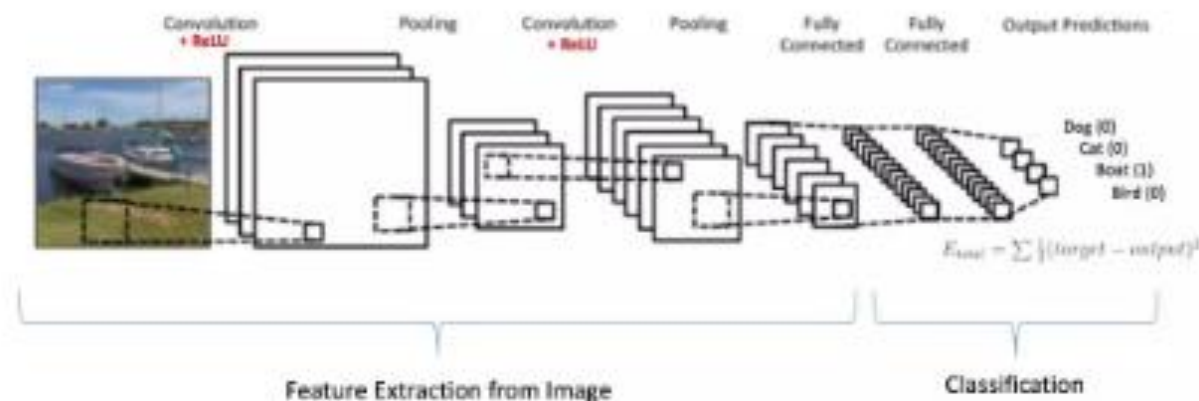
步骤1：用随机值初始化所有过滤器和参数/权重

步骤2：神经网络将训练图像作为输入，经过前向传播步骤（卷积、ReLU、池化、完全连接等步骤），得到每个类的输出概率。由于权重是随机分配给第一个训练样本，因此输出概率也是随机的。假设上面船只图像的输出概率是 $[0.2, 0.4, 0.1, 0.3]$

步骤3：计算输出层的总误差（对所有4个类进行求和）

总误差 = 目标概率和输出概率之间的差值

参考<https://www.cnblogs.com/kkx5211/p/11884267.html>



反向传播

步骤4：使用反向传播计算网络中所有权重的误差梯度，并使用梯度下降更新所有过滤器和权重/参数，以最小化输出误差。

当再次输入相同的图像时，输出概率可能就变成了 $[0.1, 0.1, 0.7, 0.1]$ ，这更接近目标向量 $[0, 0, 1, 0]$ 。

这意味着网络已经学会了通过调整过滤器、权重/参数并减少输出误差的方式对特定图像进行正确分类。

过滤器数量、大小，网络结构等参数在步骤1之前都已经固定，并且在训练过程中不会改变 —— 只会更新过滤器矩阵和连接权值。

步骤5：对训练集中的所有图像重复步骤2-4。

通过以上步骤就可以训练出卷积神经网络 —— 这实际上意味着卷积神经网络中的所有权重和参数都已经过优化，可以对训练集中的图像进行正确分类。换个角度理解为什么权重和参数要固定：有个边缘的特征过滤器，那么这个过滤器在扫描全图的时候，我们想要提取出所有的边缘区域，是不是这个过滤器不能变？如果在上一部分过滤器是边缘，到下一部分变成了平滑区域，那么在图像不同部分提取的特征就是“不伦不类”。

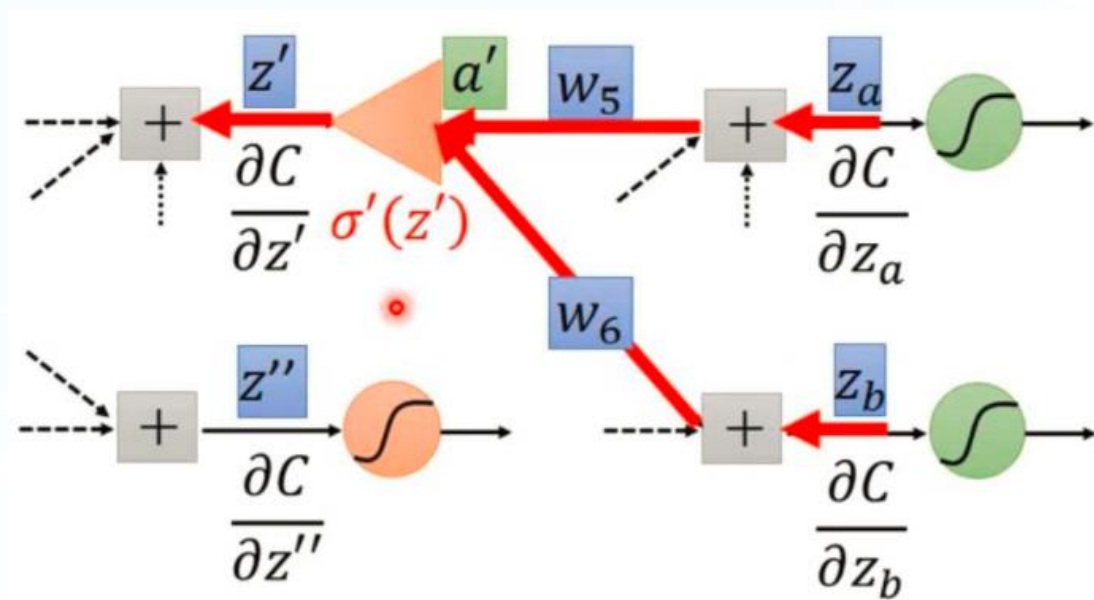
当我们给卷积神经网络中输入一个新的（未见过的）图像时，网络会执行前向传播步骤并输出每个类的概率（对于新图像，计算输出概率所用的权重是之前优化过并能够对训练集完全正确分类的）。如果我们的训练集足够大，神经网络会有很好的泛化能力，并将新图片分到正确的类别中。

梯度下降

如果把神经网络模型比作一个黑箱，把模型参数比作黑箱上面一个个小旋钮，那么根据通用近似理论 (universal approximation theorem)，只要黑箱上的旋钮数量足够多，而且每个旋钮都被调节到合适的位置，那这个模型就可以实现近乎任意功能（可以逼近任意的数学模型）。

显然，这些旋钮（参数）不是由人工调节的，所谓的机器学习，就是通过程序来自动调节这些参数。神经网络不仅参数众多（少则十几万，多则上亿），而且网络是由线性层和非线性层交替叠加而成，上层参数的变化会对下层的输出产生非线性的影响，因此，早期的神经网络流派一度无法往多层方向发展，因为他们找不到能用于任意多层网络的、简洁的自动调节参数的方法。

直到上世纪80年代，辛顿发明了反向传播算法，用输出误差的均方差（就是loss值）一层一层递进地反馈到各层神经网络，用梯度下降法来调节每层网络的参数。至此，神经网络才得以开始它的深度之旅。



梯度下降

首先明确，“正向传播”求损失，“反向传播”回传误差。同时，神经网络的每层的每个神经元都可以根据误差信号修正每层的权重。

误差 J 有了，怎么调整权重让误差不断减小？
 J 是参数 w 的函数，如何找到使得函数值最小的 w 。

梯度下降法是一种将输出误差反馈到神经网络并自动调节参数的方法，它通过计算输出误差的loss值（ J ）对参数 W 的导数，并沿着导数的反方向来调节 W ，经过多次这样的操作，就能将输出误差减小到最小值，即曲线的最低点。

梯度下降

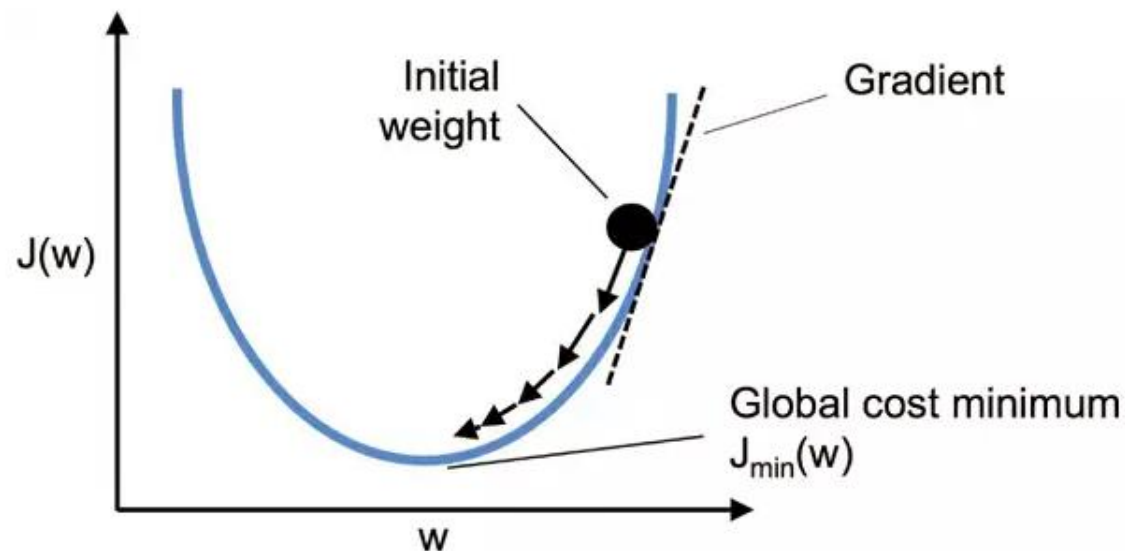


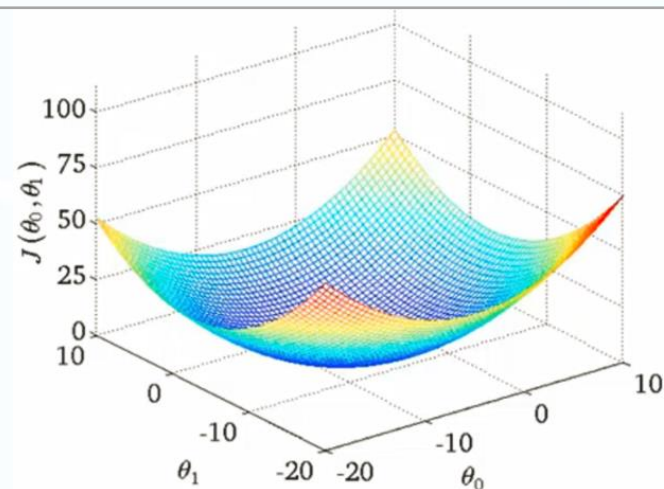
Figure 1: Gradient Decsent

梯度下降

梯度：在空间中表示函数值变化最快的方向，将上述的损失函数图像画出。

选择一组初始的参数值 θ ，就像人下山一样，我们要做的就是选择最快的下山路线，并一步一步走下山。每走一步，参数值 θ 就会变化，如何调整参数值就是梯度下降算法解决的问题。

这就是参数值更新的原理， α 乘上当前参数的偏导，整体作为更新值去作用于原参数。 α 叫做学习率，决定了下山的步子大小，当前参数的偏导则决定了下山的方向。对其中的值的大小进行讨论，随着下山接近山底，偏导的值接近于0，那么这时我们的步子是不是要迈的大一点呢，如果步子足够大，直接跨过山底走向另一个山顶呢？这就要给出一个学习率设定的策略（例如指数衰减法）。按照上述方法，不断更新参数最终达到时损失函数最小，预测值最接近真实值，这一整个过程叫做梯度下降。



Gradient descent algorithm

```
repeat until convergence {  
     $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$  (for  $j = 0$  and  $j = 1$ )  
}
```

Correct: Simultaneous update

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_0 :=$  temp0  
 $\theta_1 :=$  temp1
```